Wine Quality Classification

Luke Brace

CSC 345

30/10/2026

Source Code: Wine_Quality_Classification.ipynb

Introduction

This report compares three classifiers, k-Nearest Neighbours (k-NN), a Support Vector Machine (SVM) and Linear Discriminant Analysis (LDA), at predicting whether a white whine is good quality or poor quality based off 11 independent features.

Hyperparameters were tuned with 5 fold cross validation on the training set. On the test set, k-NN achieved the best accuracy (0.791) and the F1 score (0.845), followed by SVM with a polynomial kernel (0.767, 0.837) and LDA (0.744, 0.822). All three modules favoured the majority of the good class, and the k-NN result is likely inflated by duplicate rows in t he data set.

1.Data Preprocessing

The UCI White Wine Quality dataset contains 4898 wines, each defined by 11 numerical features, including alcohol, density and residual sugar. It also includes a quality score between 0 and 10.

1.1 Loading the data

The dataset was loaded with the pandas import. The first five rows were displayed to confirm it was being read correctly (Figure 1). Rows 3 and 4 are identical, which was the first sign that the dataset contains duplicate entries.

	fixed acidity	volatile acidity	citric acid	residual sugar	chlorides	free sulfur dioxide	total sulfur dioxide	density	pH	sulphates	alcohol	quality
0	7.0	0.27	0.36	20.7	0.045	45.0	170.0	1.0010	3.00	0.45	8.8	6
1	6.3	0.30	0.34	1.6	0.049	14.0	132.0	0.9940	3.30	0.49	9.5	6
2	8.1	0.28	0.40	6.9	0.050	30.0	97.0	0.9951	3.26	0.44	10.1	6
3	7.2	0.23	0.32	8.5	0.058	47.0	186.0	0.9956	3.19	0.40	9.9	6
4	7.2	0.23	0.32	8.5	0.058	47.0	186.0	0.9956	3.19	0.40	9.9	6

Figure 1: First five rows of the dataset

1.2 Missing Values

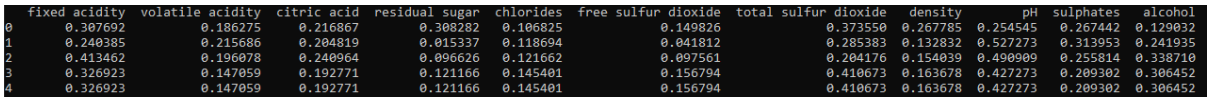
fixed acidity	0
volatile acidity	0
citric acid	0
residual sugar	0
chlorides	0
free sulfur dioxide	0
total sulfur dioxide	0
density	0
pH	0
sulphates	0
alcohol	0
quality	0

A check of every column found no missing values (Figure 2). Dropna() was kept as a backup.

Figure 2: Missing values per column

1.3 Normalisation

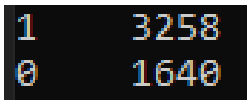
The features are on very different scales: total sulfur dioxide reaches several hundred while chlorides and density sit below sit below 1. k-NN and SVM rely on distances between points, this means unscaled features with large values would dominate. To counter this, scaling was therefore applied to rescale every feature to the range 0 – 1 (Figure 3)



	fixed acidity	volatile acidity	citric acid	residual sugar	chlorides	free sulfur dioxide	total sulfur dioxide	density	pH	sulphates	alcohol
0	0.307692	0.186275	0.216867	0.308282	0.106825	0.149826	0.373550	0.267785	0.254545	0.267442	0.129032
1	0.240385	0.215686	0.204819	0.015337	0.118694	0.041812	0.285383	0.132832	0.527273	0.313953	0.241935
2	0.413462	0.196078	0.240964	0.096626	0.121662	0.097561	0.204176	0.154039	0.490909	0.255814	0.338710
3	0.326923	0.147059	0.192771	0.121166	0.145401	0.156794	0.410673	0.163678	0.427273	0.209302	0.306452
4	0.326923	0.147059	0.192771	0.121166	0.145401	0.156794	0.410673	0.163678	0.427273	0.209302	0.306452

Figure 1: First five rows after min - max scaling

1.4 Binary Target

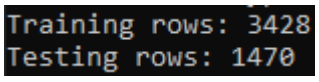


1	3258
0	1640

The quality score was converted to a binary label. Quality 6 and above become 1 (good) and anything below 6 was 0 (poor). This produced 3258 good wines and 1640 poor wines, a ration of 2:1.

Figure 4: Binary label

2.Train Test Split



```
Training rows: 3428
Testing rows: 1470
```

The data was split into 70% training (3428 rows) and 30% testing (1470) using train_test_split with a fixed random_state of 1.

Figure 5: Data split

3.Model Implementation

3.1 Models

Three classifiers were trained on the training set:

- K-Nearest Neighbours (k-NN) classifies a wine by majority vote among the k most similar wines in the training data
- Support Vector Machine (SVM) finds the boundary that separated the two classes with the widest margin.
- Linear Discriminant Analysis (LDA) projects the data onto the direction that best separates the classes.

3.2 Hyperparameter Tuning

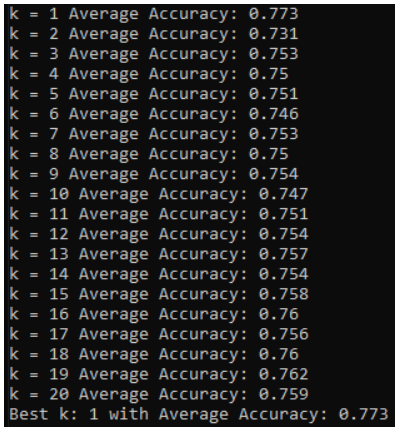


Figure 6: Accuracy for k = 1 to 20

5-fold cross-validation was used to find the best settings. The training data is split into five parts. The model is trained on four parts and tested on the fifth, and this is repeated five times so each part is used for testing once. The average of the five scores is used. The test set was not used at this stage.

k-NN**.** Values of k from 1 to 20 were tested (Figure 4). k = 1 gave the best accuracy (0.773). Accuracy then dropped at k = 2 (0.731) and slowly increased to 0.762 at k = 19.

The polynomial kernel was chosen as it scored highest. The polynomial and RBF kernels both did better than the linear kernel, which suggests good and poor wines cannot be separated with a straight line. The sigmoid kernel scored very low, so it does not suit this data.

LDA. LDA has no settings to tune.

4.Analysis

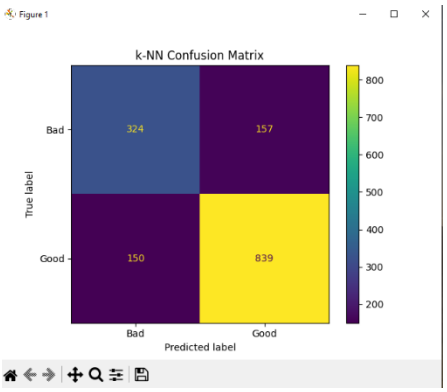
Model	Accuracy	Percision	Recall	F1 - Score
k-NN	0.791	0.842	0.848	0.845
SVM	0.767	0.789	0.891	0.837
LDA	0.744	0.771	0.881	0.822

k-NN performed best overall. It had the highest accuracy (0.791), precision (0.842) and F1 – Score (0.845). SVM came second and LDA performed the worst on every metric except recall

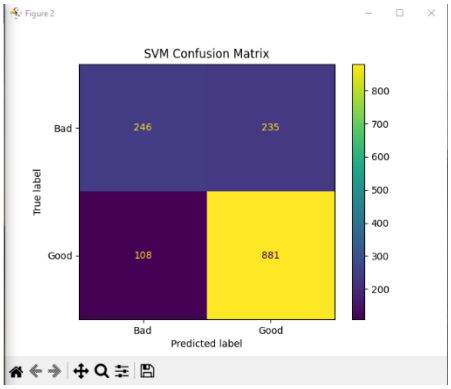
I found that the dataset was imbalanced, it had 3258 good wines and 1640 bad wines. This means a model that predicted good wines would still get 67%, The F1 – score and confusion matrices give a better look at this project.

The confusion matrices show that all three models were better at identifying good wines than bad wines. k-NN correctly identified 324 of the 481 bad wines (67%), while SVM only identified 246 (51%) and LDA 223 (46%). SVM and LDA had very high recall. I found that they predicted “good” most of the time which caused a lot of bad wines to be misclassified as good therefore lowering precision.

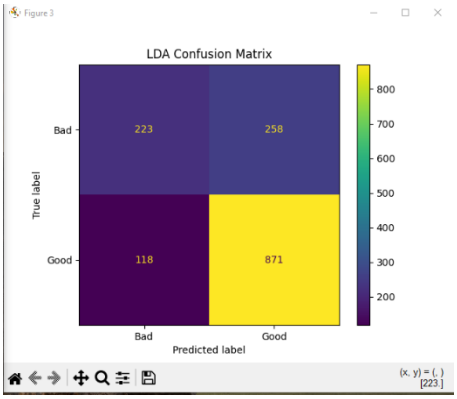
4.1 Confusion Matrices



k-NN correctly classified 324 bad wines and 839 good wines. It misclassified 157 bad wines as good and 150 good wines as bad. This is the most even amount of errors across of the other models. It handled both classes well.



SVM correctly classified 881 good wines, the most of any model. It misclassified 108 good wines as bas. However, it misclassified 235 bad wines as good wines, almost half of which are all the bad wines. This shows it favours predicting the majority.



LDA correctly classified 871 good wines but only 223 bad wines. It misclassifies 258 bad wines as good. This makes LDA the weakest model at identifying poor quality wines.

4.2 k-NN

Strengths:

- Best results overall, with the highest accuracy, precision and F1 score.
- The most balanced model, correctly identifying 67% of bad wines.
- Makes no assumptions about how the data is distributed

Weaknesses:

- Cross validation selected k = 1, this means that each wine is only classified using only its single closest neighbour. This can lead to overfitting and makes the model more sensitive
- The dataset contains duplicate rows. If the same wine appears in both, k = 1 will find the supplicate, which will make the results look better than they are.
- Because it compares each new wine, it can be slow when making prediction on large datasets

4.3 LDA

Strengths:

- Needs no hyperparameter training
- Fast and simple to train
- Works as a useful baseline

Weaknesses:

- The lowest accuracy, precision and F1 score of the three models
- Misclassified 258 of the 481 bad wines as good, more than it got right
- It assumes the classes features can be separated by a straight line and that the features are normally distributed.

4.4 SVM

Strengths:

- It had the highest recall (0.891), so it rarely missed a good wine
- The poly (0.772) and RBF (0.771) kernels both beat the linear kernel (0.752). This means that the SVM can handle the non linear boundary between good and bad wines

Weaknesses:

- Struggled with the bad wines, misclassifying 235 of the 481 as good
- The slowest model to tune
- Hard to explain why it makes each prediction

4.5 Limitations

- Duplicate rows - Duplicate rows were not removed, which inflates k-NN results.
- Single split – Results come from a one train test split. Repeating the evaluation over several random splits would give a more reliable estimate.

- Class imbalance – Class weighting could reduce the bias towards the good class in SVM and LDA

5.Conclusion

k-NN was the best performing model on this dataset, achieving the highest accuracy and F1-score. However, its selection of k = 1 and the duplicate rows in the dataset mean its results likely look better than they actually are. SVM with the polynomial kernel was a strong second and is likely to be more reliable on new data. LDA was the weakest, as its linear assumptions do not fit this dataset well. All three models were affected by the class imbalance.

Full Code:

(originally coded in VS, then copied in jupyter)

```
# Imports
import pandas as pd
from sklearn.preprocessing import MinMaxScaler

# TASK 1: Data Preprocessing

# Clean it up in command prompt
pd.set_option("display.max_columns", None)
pd.set_option("display.width", 200)

# Step 1: Load the dataset
# Step 1.1: Display the first few rows of the dataset

data = pd.read_csv("winequality-white.csv", sep=";")
print(data.head())

# Step 2: Find/Check for missing values
print(data.isnull().sum())

# Step 2.1: No missing vlalues found, drop just in case
data = data.dropna()

# Step 3: Separate features and target variable
X = data.drop("quality", axis=1)
y = data["quality"]

# Step 4: Normalise the features, all between 0 and 1
scaler = MinMaxScaler()
x = scaler.fit_transform(X)
x = pd.DataFrame(x, columns=data.columns[:-1])
print(x.head())

# Step 5: Make quality 1 = good, 0 = bad
y =[]
for q in data["quality"]:
    if q >= 6:
        y.append(1)
    else:
        y.append(0)

y = pd.Series(y)
print(y.value_counts())

# TASK 2: TRAIN TEST SPLIT

# Step 1: Split the dataset into training 70% and testing sets 30%
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(x, y, test_size=0.3, random_state=1)

# Step 2: Diplay the rows

print("Training rows:", len(X_train))
print("Testing rows:", len(X_test))

# TASK 3.1: TRAIN THE MODELS

from sklearn.neighbors import KNeighborsClassifier
from sklearn.svm import SVC
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis
```

```
# Step 1: Train k-NN (k = 5)
print("Training k-NN")
knn = KNeighborsClassifier(n_neighbors=5)
knn.fit(X_train, y_train)
print("k-NN trained, training accuracy:", round(knn.score(X_train, y_train), 3))

# Step 2: Train SVM
print("Training SVM")
svm = SVC()
svm.fit(X_train, y_train)
print("SVM trained, training accuracy:", round(svm.score(X_train, y_train), 3))

# Step 3: Train LDA
print("Training LDA")
lda = LinearDiscriminantAnalysis()
lda.fit(X_train, y_train)
print("LDA trained, training accuracy:", round(lda.score(X_train, y_train), 3))

print("All three models trained")

# TASK 3.2: TUNE THE HYPERPARAMETERS

from sklearn.model_selection import cross_val_score

# Step 1: Tune k-NN
print("Tuning k-NN")
best_k = 0
best_k_score = 0

for k in range(1, 21):
    knn = KNeighborsClassifier(n_neighbors=k)
    scores = cross_val_score(knn, X_train, y_train, cv=5)
    score = scores.mean()
    print("k =", k, "Average Accuracy:", round(scores.mean(), 3))

    if score.mean() > best_k_score:
        best_k_score = score.mean()
        best_k = k

print("Best k:", best_k, "with Average Accuracy:", round(best_k_score, 3))

# Step 2: Tune SVM
print("Tuning SVM")
best_kernal = ""
best_kernal_score = 0

for kernal in ["linear", "poly", "rbf", "sigmoid"]:
    svm = SVC(kernel=kernal)
    scores = cross_val_score(svm, X_train, y_train, cv=5)
    score = scores.mean()
    print("Kernal =", kernal, "Average Accuracy:", round(scores.mean(), 3))

    if score.mean() > best_kernal_score:
        best_kernal_score = score.mean()
        best_kernal = kernal

print("Best kernal:", best_kernal, "with Average Accuracy:", round(best_kernal_score, 3))

# Step 3: LDA has no hyperparameters to tune
print("\nLDA: no hyperparameters to tune")

# Step 4: Retrain k-NN and SVM on the full training set using the best settings
knn = KNeighborsClassifier(n_neighbors=best_k)
knn.fit(X_train, y_train)

svm = SVC(kernel=best_kernal)
```

```
svm.fit(X_train, y_train)

print("\nFinal models trained with k =", best_k, "and kernel =", best_kernal)

# TASK 4: EVALUATE THE MODELS ON THE TEST SET

from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay
import matplotlib.pyplot as plt

# Step 1: Evaluate k-NN
y_pred = knn.predict(X_test)
print("\nk-NN results:")
print("Accuracy:", round(accuracy_score(y_test, y_pred), 3))
print("Precision:", round(precision_score(y_test, y_pred), 3))
print("Recall:", round(recall_score(y_test, y_pred), 3))
print("F1-Score:", round(f1_score(y_test, y_pred), 3))

cm = confusion_matrix(y_test, y_pred)
print("Confusion matrix:")
print(cm)
ConfusionMatrixDisplay(cm, display_labels=["Bad", "Good"]).plot()
plt.title("k-NN Confusion Matrix")

# Step 2: Evaluate SVM
y_pred = svm.predict(X_test)
print("\nSVM results:")
print("Accuracy:", round(accuracy_score(y_test, y_pred), 3))
print("Precision:", round(precision_score(y_test, y_pred), 3))
print("Recall:", round(recall_score(y_test, y_pred), 3))
print("F1-Score:", round(f1_score(y_test, y_pred), 3))

cm = confusion_matrix(y_test, y_pred)
print("Confusion matrix:")
print(cm)
ConfusionMatrixDisplay(cm, display_labels=["Bad", "Good"]).plot()
plt.title("SVM Confusion Matrix")

# Step 3: Evaluate LDA
y_pred = lda.predict(X_test)
print("\nLDA results:")
print("Accuracy:", round(accuracy_score(y_test, y_pred), 3))
print("Precision:", round(precision_score(y_test, y_pred), 3))
print("Recall:", round(recall_score(y_test, y_pred), 3))
print("F1-Score:", round(f1_score(y_test, y_pred), 3))

cm = confusion_matrix(y_test, y_pred)
print("Confusion matrix:")
print(cm)
ConfusionMatrixDisplay(cm, display_labels=["Bad", "Good"]).plot()
plt.title("LDA Confusion Matrix")

# Step 4: Show all three confusion matrix plots
plt.show()
```